MLPerf EDU

A Laptop-Scale, Quality-Gated Benchmark Suite for ML Systems

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is taught almost everywhere as a set of numbers to read and almost nowhere as a discipline to practice. The reason is practical. Production suites such as MLPerf teach that discipline properly, holding task, data, metric, and quality target fixed so that two results can be compared at all, but their compute and submission requirements put them out of reach of a course or a single reviewer. The usual substitute is a collection of small models, which solves the resource problem and quietly discards the quality gate. A student can then report a faster run after weakening a target, dropping a validation split, or timing a warmup pass, and nothing in the setup objects.

MLPerf EDU is an attempt to keep both properties at once, an executable benchmark specification that runs on a laptop without giving up the gate. The central decision is that no target is ever set by this project. Each workload inherits its task, dataset, metric, and quality target from an upstream authority, and a result is admitted only after the inherited contract is met. That rule is deliberately uncomfortable, because it guarantees the suite reports its own failures.

It does. The v0.1 registry holds 14 workload identities across 7 suites, all of which run locally. On one Apple M5 Max laptop the suite executed 14 inherited contracts and reproduced 8 of them. The 6 shortfalls are recorded as misses rather than rescued by a post-hoc tolerance, and the workloads carrying them are held out of score-bearing review by the same rule that admitted the rest. Of the 9 cases that rule admits, 8 meet their gate, with reference medians spanning 0.280 seconds to 35.1 minutes and totaling 62.0 minutes for one pass through the score-bearing suite. Three execution profiles separate installation checks, comparable classroom runs, and controlled research variants without encoding optimizations into workload identities.

The result is a benchmark artifact whose score-bearing path a student can run, inspect, and argue with in an afternoon. It makes no claim to production-scale equivalence, and these are project measurements rather than MLCommons-verified results. The path to that status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Machine-learning systems now span training, serving, retrieval, graph analytics, time-series forecasting, vision, and embedded inference. Comparing those systems requires more than timing a script. A benchmark must hold task semantics and quality constant while recording the model, data, hardware, scenario, and measurement procedure. MLPerf established this discipline for production systems (Mattson et al., 2020; Reddi et al., 2019; Banbury et al., 2021). The same discipline is valuable in education, yet production-scale workloads and submission infrastructure often exceed the time, hardware, and operational budget of a course or an individual artifact review.

The usual educational substitute is a collection of small models. Small models solve the resource problem, but they do not automatically teach benchmarking. A student can report a faster run after changing the data, silently weakening quality, measuring warmup, or using a synthetic shortcut. None of those results supports a fair systems comparison. The missing learning objective is *Benchmarking Literacy*, the ability to construct, inspect,

reproduce, and defend an evidence-backed performance claim.

MLPerf EDU addresses that gap with an executable specification rather than a static scoreboard. Its scope is locally executable, quality-gated evaluation for teaching and studying single-node ML systems. The suite supports controlled work on processors, memory systems, runtimes, compilers, and model execution while excluding distributed and datacenter-scale claims. Backward design begins with the activity a student or researcher must perform, such as reproducing a result, isolating a bottleneck, changing one systems variable, and defending comparability. Workloads are admitted only when an established task can support that activity on local hardware without replacing its model, data, evaluator, or quality gate with a project-defined proxy.

Definition 1 (Evaluation Tuple)**.** *A benchmark configuration $\mathcal{S} = (W, D, M, H, S, C, P)$ binds the workload, data, model, hardware, scenario, constraints, and measurement protocol. It produces $\mathcal{M} = (Q, L, T, E)$, containing task quality, latency, throughput, and an energy observation when calibrated. An evidence package $\mathcal{E}$ records the configuration, measurements, source state, assets, and model lineage. Two results are comparable only when $\mathcal{E}$ makes every controlled equality and intentional difference explicit.*

The framework turns the tuple into a Fail-Closed Admission Rule. A result r is admissible only when

$$\begin{aligned}\text{admit}(r) = {} & G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \\ & \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r),\end{aligned}$$

where the semantic gate is task quality for score-bearing cases and functional correctness plus model lineage for performance-bearing cases. The remaining terms check the declared protocol, content-bound evidence, and stable host conditions. Admissibility and evidence strength are deliberately separate. A case may pass every gate while remaining provisional until its repeatability protocol is complete.

Four terms recur throughout the paper and are worth fixing here, since the distinctions they draw are what the Admission Rule enforces. A *score-bearing* case is one whose result is decided by task quality against an inherited target, so it can carry a quality claim. A *performance-bearing* case is one whose result is decided by functional correctness and verified model lineage, so it can carry a timing or throughput observation but not a quality claim. A *promotion-scope* workload is one whose contract is complete enough that a conformant result would be eligible for a published baseline, as opposed to a *functional-stage* workload, which executes end to end but is held back because its quality path does not yet conform. *Fail-closed* means the default is exclusion. A case is admitted only by satisfying every gate, and a case that satisfies none of them by failing silently is treated as a failure rather than as missing data.

The paper makes six contributions.

1. **An executable benchmark contract.** The Evaluation Tuple and Fail-Closed Admission Rule become registry, runner, grading, host-state, and reporting invariants rather than paper-only guidance.
2. **A backward-designed workload portfolio.** 14 workload identities cover language, retrieval, recommendation, reinforcement learning, graph learning, time series, vision, and embedded inference. Of these, 5 are admitted only at a functional setup stage rather than through weaker targets or invented quality claims.
3. **Three execution intents.** The `min`, `max`, and `pro` profiles separate setup checks, comparable classroom runs, and research variants while preserving stable workload identities.
4. **Auditable and lineage-bound evidence.** Runs emit JSON, HTML, CSV, and provenance manifests that bind source, assets, hardware, metrics, and training checkpoints.
5. **A measured basis for single-run acceptance.** A determinism study over 36 executions shows inference quality is bit-identical across seeds and across the MPS and CPU backends, which justifies accepting a quality verdict from one run, and shows that training is not, which refutes it. In both training workloads swept, the seed spread exceeds the margin to the gate and the verdict flips.
6. **Measured laptop references.** 12 source-locked cases traverse the public command path, and 8 of the 14 executed contracts reproduce their inherited target.

The goal is not to replace official MLPerf suites or imply MLCommons approval. It is to make professional evaluation practice inspectable at a scale students and researchers can run locally. Figure 1 summarizes the system.

The paper follows this contract from gap to evidence. Section 2 positions the work against existing benchmark surfaces and shows why none of them could be adopted directly. Section 3 states the three design principles that keep a laptop-scale workload faithful to its upstream task, and Section 4 describes the implementation that turns those principles into registry, runner, grading, and validation invariants. Section 5 defines the workload portfolio one benchmark at a time, naming for each what it measures, why it was admitted, where its target came

from, and which evaluator decides it. Section 6 develops the three execution profiles that separate setup, comparable, and research runs. Section 7 reports the measured evidence, Section 8 the classroom progression built on the same contract, and Section 9 what the evidence does not yet establish.

2 Related Work

Before describing a new benchmark suite it is worth establishing why an existing one could not simply be adopted. Benchmarking is a mature field with well-governed suites, and the default assumption should be that a course uses one of them. This section reviews the surfaces that were candidates and identifies the specific property each one gives up. The pattern that emerges is consistent enough to state in advance. Suites that hold task quality fixed require compute budgets and submission infrastructure a course does not have, and suites that fit a course's budget do so by dropping the quality gate, which is the property that makes results comparable in the first place.

MLPerf. The official MLPerf Training and Inference suites established quality-constrained, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). Their rules make quality, scenarios, measurement regions, and disclosure part of result validity rather than reporting conventions (MLCommons, 2026). MLPerf Tiny extends the methodology to constrained embedded systems (Banbury et al., 2021). MLPerf EDU inherits that discipline and selected workload contracts while targeting locally executable teaching and single-node research. It does not claim that laptop execution reproduces cluster or microcontroller scale; instead it asks what stays valid when the compute budget shrinks to a laptop and the reviewer is an instructor rather than a submission committee. Table 1 summarizes the positioning.

DAWNBench. DAWNBench introduced time-to-accuracy and cost-to-accuracy (Coleman et al., 2019). Its central methodological contribution was to connect system performance to a fixed task-quality threshold. MLPerf EDU retains that quality-first rule but differs in scope rather than in commitment. It targets one fixed classroom exercise with an inherited gate, not a leaderboard optimized for cost or wall-clock time across cloud providers.

TorchBench. TorchBench provides broad PyTorch performance-regression coverage for framework development (Ansel et al., 2024). MLPerf EDU shares its executable PyTorch surface but adds task datasets, inherited quality gates, classroom profiles, grading, checkpoint lineage, and a deliberately small governed portfolio.

Table 1. Comparison with established benchmark surfaces. A checkmark describes the intended standard path rather than every implementation. The bottom row is the one no existing surface occupies: a complete single-laptop path that still holds an inherited quality gate.

	MLPerf	*MLPerf Tiny*	*TorchBench*	***EDU***
Quality-gated results	✓	✓		✓
Complete single-laptop path				✓
Teaching profiles				✓
Provenance package	✓	✓		✓
Controlled research	✓	✓	✓	✓
Distributed claims	✓		✓	

Breadth is therefore not the objective. Each admitted workload must support a complete semantic and evidentiary contract. Where TorchBench asks whether a kernel or graph transformation regressed, MLPerf EDU asks whether a student's reported comparison is admissible under a declared task and quality contract; the two questions are complementary, not competing.

Educational ML frameworks. Framework-building courses make tensors, operators, and automatic differentiation inspectable. MLPerf EDU is complementary. Its learning object is the benchmark contract rather than the framework implementation. PyTorch supplies an immediate standard execution path without coupling the specification to a custom educational framework. Other frameworks can later implement the same contracts, but they are not required to define them. A student who has built a framework from primitives still needs a second discipline, knowing when a measured result is comparable to another; MLPerf EDU supplies that second discipline without requiring the first.

Non-ML benchmarks. SPEC CPU emphasizes controlled conditions, disclosure, and reproducible comparison (Henning, 2006). TPC shows that benchmark governance and multi-dimensional reporting are as important as the workload code (Poess and Floyd, 2000). MLPerf EDU applies those lessons to a domain where quality and task semantics, not only raw throughput, complicate the same discipline. A classroom result that is fast but silently below the accuracy target is no more admissible than a SPEC run compiled outside the reference flags.

The table exposes a gap rather than a crowded field. Suites that hold quality fixed run above the resource envelope of most courses and individual reviewers; suites that fit that envelope typically drop the quality gate rather than shrink the task. MLPerf EDU is designed to occupy the one row that keeps both properties at once.

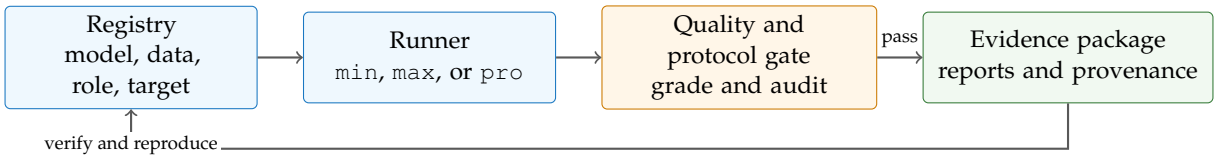

Figure 1. MLPerf EDU system architecture. Workloads flow through the unified CLI, asset, runner, grading, reporting, and provenance layers. The Evaluation Tuple binds each result to its model, data, hardware, scenario, constraints, and measurement protocol.

That gap is what forced a build rather than an adoption. The missing artifact is not another collection of small models, which is abundant, but a suite in which the quality gate survives the reduction in scale. Nothing prevents a laptop-scale suite from inheriting an upstream target; what prevents it is that inheriting the target means accepting that some workloads will miss it, and a suite built to demonstrate its own workloads has an incentive to calibrate targets to what it can reach. The remainder of the paper describes a design that removes that incentive by construction, and reports what happens when the resulting contracts are run on one laptop, including where they fail.

3 Design Principles

Four principles guide MLPerf EDU's design, each addressing a failure mode that turns a runnable script into an invalid benchmark. The first is load-bearing for the other three.

3.1 Inherited Targets, Never Invented Ones

Every quality target in the suite is inherited from an upstream authority, and no target is ever set by this project. The reason is an incentive problem rather than a matter of taste. A suite that defines its own thresholds will tend to define ones it can meet, and it will do so without any individual choice looking dishonest, because each threshold can be justified locally as reasonable for the scale. The result is a benchmark that cannot fail, which is also a benchmark that cannot inform.

Inheritance removes the discretion that makes that drift possible. A target arrives from a paper, a leaderboard, an official evaluator, or a published model card, together with the dataset split and metric it was computed on, and the project's only remaining decision is whether it is met. When a laptop implementation falls short, the shortfall is reported as a miss. Nothing in the framework can convert it into a pass, because the number it would have to change belongs to someone else.

This has a cost, and the cost is the point. In this paper, 6 contracts miss their inherited targets, and each one would have been trivially removable by nudging a threshold or widening a tolerance. Keeping them is what makes the passing results mean anything, and it is also what gives students a suite in which failure is a normal, inspectable outcome rather than a sign that they ran it wrong.

3.2 Pedagogical Transparency Over Performance

Production benchmarks optimize for throughput with custom CUDA kernels, mixed-precision arithmetic, and operator fusion. Those optimizations can make the computation invisible to students. MLPerf EDU inverts this priority for its reference workloads. The standard execution path uses inspectable Python and PyTorch with explicit operations and named dimensions. Thin adapters aim to preserve the authoritative model and task contract rather than replacing it with a project-designed approximation. Conformance is audited rather than assumed. The current keyword-spotting adapter preserves the DS-CNN topology and quality gate but is prediction-nonidentical to the pinned LiteRT graph, so promotion is blocked. Students can still inspect its depthwise and pointwise convolutions as separate `nn.Conv2d` layers. The reference path is a readable point of comparison, not a performance ceiling. Optimized kernels and alternative runtimes belong in recorded `pro` configurations and remain subject to the same semantic gate.

3.3 Real Data with Provenance

A benchmark without real data teaches stress testing, not systems evaluation. Students must observe how data distributions affect convergence, accuracy, and ranking. Table 3 records every dataset used by a canonical mode, while the executable registry carries structured dossiers with pinned sources, sizes, splits, licenses, release status, and digest policy. Fetching and redistribution are separate decisions. A dataset can support a reproducible local run while its bytes remain excluded from a public evidence package.

```yaml
workload:
  id: causal-language-modeling
  suite: language
  profiles: [min, max, pro]
  runner: nanogpt
  dataset: tinyshakespeare
  quality_target:
    metric: cross_entropy_loss
    direction: lower
    threshold: 1.4697
    target_basis: literature
  public_status: experimental
```

Listing 1. Registry-driven workload metadata. Workload rows declare the user-facing selector, suite, profile coverage, assets, and quality policy.

3.4 Complete Train–Checkpoint–Inference Loop

Many tools provide only training or only inference. MLPerf EDU preserves the complete pipeline when model lineage is part of the workload contract. Causal language-model training produces a quality-approved checkpoint. Full, prefill, and decode inference load that checkpoint and verify its report, manifest, and digest before measurement. The current registry records dataset, model, checkpoint, profile, quality, and result-role metadata in one place, ensuring deterministic, reproducible experiments. Training, full generation, prefill, and decode remain modes or phases of one workload. Treating them as separate workload identifiers would hide their shared task and checkpoint lineage.

4 Architecture and Implementation

The implementation follows the original white-box design. A declarative registry selects assets, runners, scenarios, profiles, gates, evidence roles, and promotion rules. A unified command-line interface drives fetch, audit, run, grade, verify, report, package, and validation operations. The same registry generates the flat `workloads.yaml` compatibility mirror, website tables, and paper snapshot. This single-source design makes disagreement between the paper and executable contract a build failure rather than a documentation bug.

4.1 Unified Dataset Factory

Dataset recipes distinguish bundled, fetched, cached, synthetic, and restricted assets. Reports record the resolved mode and relevant digests. A `min` runner may use a reduced or synthetic input, but the public audit rejects that mode for score-bearing evidence. Dataset dossiers also record citation, license, release status, and expected cache behavior.

4.2 Model Registry

Model records bind a user-facing identifier to its implementation or pinned upstream revision. Checkpoint-backed inference adds a lineage record containing the source training workload, source task quality, and checkpoint digest. This prevents a serving result from silently changing weights between repetitions.

4.3 Load Generator and Inference Harness

The Python-native load generator implements inspectable scheduling for single-stream, offline, server, and multi-stream experiments (Reddi et al., 2019). It records warmup and measurement counts, latency samples and percentiles, throughput, and functional outcomes. It is designed for teaching and local experimentation. It is not a drop-in replacement for official MLPerf LoadGen.

4.4 Power Measurement

Promotion campaigns record power source, Low Power Mode, and sleep or wake state at each execution boundary on supported hosts. A campaign refuses to start without external power and invalidates an affected attempt when the host state changes. These guards improve timing control but do not provide calibrated energy measurement. Absolute joule claims still require supported platform counters or an external meter with a disclosed calibration.

4.5 Submission Artifacts

Each run produces a JSON report and human-readable HTML and CSV views. A `.provd.json` manifest binds the report and declared sidecars by size and SHA-256 digest. Hardware and software fingerprints, asset dossiers, quality metadata, and checkpoint lineage travel with the result. Packages rewrite machine-specific absolute paths so that verification works after extraction in a clean directory.

Hashing establishes integrity, not authenticity. A self-consistent manifest can still be fabricated by its creator. Independent reruns, signed attestations, or controlled execution are needed for stronger trust. The framework deliberately states this boundary instead of claiming that checksums prevent fraud. The manifest therefore provides unauthenticated integrity, not proof of origin or execution.

4.6 CLI as Evaluation Tuple Instantiation

The CLI makes the Evaluation Tuple concrete. Suite, workload, variant, and profile select the model, data, scenario, constraints, and protocol. The commands below cover the normal review path.

```
$ mlperf doctor
$ mlperf list workloads
$ mlperf fetch \
    --workload causal-language-modeling \
    --mode training --profile max
$ mlperf run \
    --workload causal-language-modeling \
    --mode training --profile max
$ mlperf grade submissions/
$ mlperf verify submissions/run.provd.json
$ mlperf package submissions/run.provd.json
$ mlperf audit --policy public
```

Listing 2. CLI path from setup to verified evidence.

The same metadata drives individual runs and release presets. This reduces the chance that a paper-only configuration diverges from the executable path.

4.7 Validation Methodology

Validation is layered because a single green status can hide different kinds of failure. Unit and integration tests exercise runners, schemas, registry invariants, grading, provenance, packaging, and generators. Negative tests reject path traversal, changed source bytes, seed disagreement, incomplete functional gates, partial batch output, unavailable devices, unresolved dataset bytes, and timing evidence above its declared variation limit. The `smoke` preset runs four representative `min` workloads. `coverage` runs all 14 `min` workloads. `max` selects canonical quality paths for the 9 promotion-scope workloads and bounded functional probes for the other 5. `pro` selects the research workloads and repeats their `max` contracts under a separate profile. Each phase verifies provenance and applies its declared quality or functional gate.

The full artifact test suite passes with two expected skips. The smoke and coverage presets passed 4/4 and 14/14 workloads. A retained one-measurement `pro` validation passed the four promotion-scope research workloads, their quality gates, provenance verification, and grading with zero warnings. The expanded `pro` plan adds the five bounded functional probes, whose individual `max` paths also pass. The 12 source-locked `max` cases all have retained evidence. A clean Python 3.12.13 wheel installed outside the checkout, loaded 14 workload definitions and 12 evidence records, executed the starter smoke path, and verified the resulting provenance.

The Quarto site renders 31 pages and passes 62 desktop and narrow-viewport checks. The paper generator binds this manuscript to the same evidence index and source lock. The strict public audit remains fail-closed because all 14 workloads are experimental. The validation supports internal consistency and reviewability, not external reproducibility. Hosted Linux CI, dataset-policy decisions, and independent reproduction remain separate gates.

5 Workload Suite

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on the target platform. No workload is environment-gated, so every contract in this section can be run and checked by a reader with the same hardware class. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. Modes, phases, execution options, and research variations do not create new workload identities. This registry is the source of truth. The paper imports its counts and evidence table from a generated snapshot, which prevents prose from drifting away from executable metadata.

Each workload is run under one of three execution profiles, `min`, `max`, and `pro`, which separate installation checks from comparable runs and research variants. Section 6 develops that separation and the reason it is a property of the run rather than of the workload.

The registry also separates public interpretation from execution success. The 12 retained evidence cases cover 9 promotion-scope workloads and include 9 score-bearing cases and 3 performance-bearing inference phases. The other 5 workloads have end-to-end functional evidence only. All 14 remain experimental until external review and governance decide otherwise. A successful execution can therefore anchor a lab or research comparison without being presented as an official benchmark score. This fail-closed distinction is important for small workloads because functional smoke data and reduced teaching exercises are not quality baselines.

Selection principles. Admission requires five properties. The task must have a recognizable upstream authority, fit the declared laptop envelope, preserve the system behavior it claims to expose, support a controlled pedagogical intervention, and provide an evaluator with a defensible gate. Resource accessibility alone is insufficient. A small model is rejected when its data, evaluator, or target must be invented for this project. Behavioral fidelity also does not imply production-scale equivalence. The framework makes no roofline or bottleneck classification without measurements, and a taxonomy field with no supporting measurement is left blank rather than estimated.

Alignment with MLCommons working groups. To ensure comprehensive coverage of modern machine-learning systems, the portfolio maps directly onto eight core MLCommons working group domains. Each suite

Table 2. Current workload contracts, generated from the executable registry. Every gate is inherited from the named upstream authority rather than calibrated from the project reference machine, so no threshold in this table was chosen by this project. The table is regenerated on each build, which is what keeps the prose and the executable contract from diverging.

Workload	Upstream Authority	Mode	Gate
`anomaly-detection`	MLCommons MLPerf Tiny, ToyADMOS, and DCASE	inference	ROC AUC $\geq$ 0.8500
`causal-language-modeling`	nanoGPT upstream repository	training; inference (full, prefill, decode)	loss $\leq$ 1.470
`code-generation`	Qwen and EvalPlus	inference	HumanEval+ pass@1 $\geq$ 0.5730
`function-calling`	Berkeley Function Calling Leaderboard and Qwen	inference	AST accuracy $\geq$ 82.92%
`graph-node-classification`	Open Graph Benchmark	training	test acc. $\geq$ 71.74%
`image-classification`	MLCommons MLPerf Tiny	inference	top-1 $\geq$ 85.00%
`image-generation`	NVIDIA EDM reference implementation	inference	FID $\leq$ 1.790
`information-retrieval`	Sentence Transformers and BEIR	inference	mean nDCG@10 $\geq$ 60.72%
`keyword-spotting`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 90.00%
`recommendation`	MLCommons MLPerf Training v0.5 recommendation	training	hit_rate_at_10 $\geq$ 0.6350
`reinforcement-learning`	Historical MLPerf Training MiniGo	training	move prediction $\geq$ 0.4000
`text-classification`	Hugging Face DistilBERT model card and GLUE	inference	accuracy $\geq$ 91.06%
`time-series-forecasting`	PatchTST authors and ETTDataset	training	test MSE $\leq$ 0.2900
`visual-wake-words`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 80.00%

exposes distinct systems characteristics and pedagogical learning objectives.

1. **Edge and Tiny (MLPerf Tiny):** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depthwise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.
2. **Language and LLM Serving (MLPerf LLM):** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.
3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents):** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.
4. **Graph Neural Networks (MLCommons Graph):** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on ogbn-arxiv.
5. **Time-Series and Scientific ML (MLCommons Science):** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.
6. **Recommendation Systems (MLPerf RecSys):** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.
7. **Generative AI and Diffusion (MLCommons Generative):** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.
8. **Reinforcement Learning and Games (MLPerf RL):** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

The laptop envelope. Because admission depends on it, the envelope is stated rather than left implicit. A workload qualifies when it runs to completion on a single consumer machine with no accelerator beyond an integrated or laptop-class GPU, requires no manually licensed data, keeps its largest single asset small enough to fetch over an ordinary connection, and holds its working set inside the memory such a machine ships with. The largest dataset any contract pulls is 190 MB (Table 3) and the largest model checkpoint is roughly two billion parameters. That envelope is a constraint on admission, not a measured portability claim. Every timing number in this paper comes from one machine (Section 7), and the floor of the envelope, meaning the least capable machine on which the suite still completes, is untested and remains open work.

The rest of this section takes the workloads one at a time. Each paragraph answers the same four questions, because those are the questions that decide whether a reported number means anything. What is the benchmark? Why is it in the suite, given that resource fit alone is not a reason to admit a task? Where did its quality target come from? And which metric and evaluator decide pass or fail? Exact gate values and upstream authorities are listed in Table 2 and generated from the registry, so the prose below describes the provenance of each target rather than restating its digits.

Table 3. Dataset provenance and access. The table is generated from the registry dossiers. *Review* means the data can be fetched for a local run but is not bundled in a public package.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTm1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

5.1 Language and Retrieval

Causal language modeling. Character-level next-token prediction on Tiny Shakespeare using the nanoGPT training recipe (Karpathy, 2022). It is the suite's anchor workload because it is the only one that exposes training and all three inference phases, full, prefill, and decode, under a single identity, which lets a student watch one checkpoint move through the whole lifecycle. Training produces the quality-approved checkpoint that the inference phases then consume, so a degraded training run cannot silently produce a fast inference number. The target is the validation loss nanoGPT's published recipe reaches on the same split, and the metric is that validation loss.

Text classification. Binary sentiment classification with DistilBERT on the GLUE SST-2 validation split (Sanh et al., 2019). It contributes encoder-style inference, a bidirectional attention pattern with a fixed sequence budget and no autoregressive loop, which behaves differently from the decoder workloads under batching. The target is the accuracy DistilBERT's own model card reports for SST-2, inherited rather than measured here, and the metric is top-1 classification accuracy on the official validation split.

Information retrieval. Cross-encoder reranking with a compact MS MARCO model over the three-dataset NanoBEIR subset (Reimers and Gurevych, 2019). Retrieval is included because its cost structure is unlike single-pass classification. Every query is scored against many candidates, so the workload is dominated by a rerank loop whose length is a property of the task rather than of the model. The target is the published nDCG for the same model on the same subsets, and the metric is mean nDCG@10.

Code generation. Function synthesis with Qwen2.5-Coder-0.5B evaluated on the complete 164-task HumanEval+ release (Hui et al., 2024). It is admitted because it is an execution-scored task rather than a similarity-scored one, which means quality is decided by running generated code against tests instead of comparing strings. The runner binds the full task release, Qwen's own prompt and stop rules, pinned model bytes, and a containerized EvalPlus evaluator. The target is the pass rate Qwen publishes for this model, and the metric is HumanEval+ pass@1. The recorded run reached 55.49% against a published 57.30%, so the workload is held at the functional stage.

Function calling. Structured tool-call generation with Qwen3-1.7B over the 1,150-case non-live AST split of the Berkeley Function Calling Leaderboard (Yan, 2025; Berkeley Function-Calling Leaderboard, 2026). It represents the agentic serving path, where the model's output is parsed as a structured call rather than read as text, and correctness is a property of the parsed abstract syntax tree. The target is the leaderboard's published AST summary score for this model, and the metric is that AST accuracy. The recorded run reached 78.52% against a published 82.92%, which likewise holds promotion closed.

5.2 Graph and Time Series

Graph node classification. A three-layer GCN trained on ogbn-arxiv using the official Open Graph Benchmark recipe and evaluator (Kipf and Welling, 2017; Hu et al., 2020). It is in the suite because it exposes sparse gather and scatter behavior that no dense language or vision workload produces, which makes it the only workload where memory access pattern rather than arithmetic dominates the profile. The target is the accuracy the OGB leaderboard reports for the reference GCN, and the metric is test accuracy computed by the official evaluator on the official time split. The contract carries OGB's published standard deviation as its tolerance.

Time-series forecasting. Multivariate long-horizon forecasting with PatchTST on the official ETTm1 split (Nie et al., 2023). It adds a temporal workload whose long input window, patch extraction, and multivariate attention give an attention cost that scales with history length rather than with vocabulary. The target is the test error reported in the PatchTST paper for this dataset and horizon, and the metric is test MSE.

The paper reports that figure with a standard deviation, which the contract carries as its tolerance.

5.3 Vision, Audio, and Embedded

The next four workloads inherit MLPerf Tiny task contracts (Banbury et al., 2021), which supply both the model and the official evaluation subset. Each implementation maps the official model into an equivalent PyTorch graph and adds measurement and provenance without changing the task, so the targets remain the ones MLPerf Tiny already publishes.

Image classification. ResNet8 on the official 200-example CIFAR-10 accuracy set. It is the smallest complete vision contract in the suite and serves as the reference point for embedded-scale inference. The target is the MLPerf Tiny closed-division accuracy requirement, and the metric is top-1 accuracy.

Keyword spotting. A DS-CNN over the official 1,000-example speech-command accuracy set (Warden, 2018). It contributes an audio front end, so the measured path includes feature extraction rather than starting at a ready tensor, which is where a naive harness usually underreports latency. The target is the MLPerf Tiny accuracy requirement, and the metric is top-1 accuracy.

Visual wake words. MobileNetV1 at 0.25 width on the official 1,000-example person-detection set. It represents the always-on binary vision task that motivates the tiny power envelope, where the operating point is set by a duty cycle rather than by throughput. The target is the MLPerf Tiny accuracy requirement, and the metric is top-1 accuracy.

Anomaly detection. A ToyCar autoencoder with the official log-mel feature recipe over the 248-recording accuracy index. The metric choice is the point of including it. Reconstruction error alone does not establish useful anomaly ranking, so the contract inherits MLPerf Tiny's ROC AUC evaluator, which scores the ranking rather than the reconstruction. The target is the MLPerf Tiny ROC AUC requirement.

Image generation. Diffusion sampling with NVIDIA EDM on CIFAR-10 (Karras et al., 2022). It is the only generative vision contract and the only one whose cost is set by sampler step count, which makes step count a systems variable a student can vary without touching the model. The target is the FID reported in the EDM paper for this configuration, and the metric is FID over the official three-trial, 50,000-image procedure. The recorded run reported 1.80 against a published 1.79.

5.4 Recommendation and Reinforcement Learning

Recommendation. Neural collaborative filtering on MovieLens-20M, inherited from the retired MLPerf Training v0.5 recommendation contract (He et al., 2017; Harper and Konstan, 2015). This workload reaches the laptop through a different route than the others, and the substitution is worth stating plainly. The current DLRM contract requires manually licensed Criteo Terabyte data, a roughly 90 GB checkpoint, and a 256 GB-class runtime, none of which fits the declared envelope (Naumov et al., 2019). The v0.5 contract trains on a laptop while keeping a published target. That target is the 0.635 HR@10 specified by the MLPerf Training v0.5 rules, evaluated leave-one-out against 999 sampled negatives per user, and the metric is that hit rate. The first recorded run reached 0.623 and is recorded as a miss rather than rescued by relaxing the target or the candidate count, either of which would make the published number trivially beatable. The substitution is not an equivalence. NCF exercises embedding lookup and dense interaction, but its categorical cardinality is far below Criteo's, so it does not expose the memory-capacity wall that motivated DLRM.

Reinforcement learning. Self-play training of a joint policy and value network under the MLPerf Training MiniGo contract (Mattson et al., 2019). It is the only workload whose training data does not exist before the run, since self-play generates its own trajectories, which makes it the suite's one example of a workload where the data pipeline and the model are the same system. A PyTorch adapter runs the pinned upstream reference so that Go rules, tree search, the self-play loop, and professional-move evaluation execute unmodified from a hash-validated archive. The target is the professional-move prediction gate from the original 9-by-9 MiniGo contract, and the metric is that move-prediction accuracy. The laptop configuration reduces games per generation and generation count against the reference, so it remains a bounded probe and quality conformance stays open.

Optimization methods such as batching, speculative decoding, and tool scheduling remain configurations or research experiments rather than separate workload identities.

6 Execution Profiles

A benchmark suite that serves both a first-week installation check and a term-long architecture project has to distinguish those runs without letting either redefine the task. MLPerf EDU does that with three execution profiles. They describe execution *intent*, not difficulty,

Table 4. Execution profiles and their intended interpretation. The profile describes why a run was performed, not how hard it is. Workload identity, model, data, evaluator, and gate are identical across all three, so moving between profiles changes what may be claimed from a result rather than what was measured.

Profile	Interpretation
`min`	Fast correctness and setup path. No public score.
`max`	Comparable reference path with the declared assets and gates.
`pro`	Research variants and ablations with configuration-dependent comparability.

and they are a property of the run rather than of the workload, so the same workload identity, model, data, evaluator, and gate persist across all three.

`min` is a quick installation and plumbing check. It may use reduced or synthetic inputs and never supports a public score. `max` is the standard comparable path with the declared model, real data, gate, and protocol for promotion-scope workloads; for a functional-stage addition it remains a bounded setup probe that cannot support a quality or performance claim. `pro` is the research envelope for ablations, backend studies, repetitions, and user-controlled variants, and a `pro` result is comparable only when its recorded configuration defines the comparison.

The reason for three profiles rather than one is that the alternative encodes optimization into workload names. A suite without this separation accumulates identities such as `resnet-batched` or `gpt-compiled`, at which point the workload no longer denotes a task and cross-result comparison becomes a question about naming conventions. Keeping batching, compilation, precision, scheduling, and caching inside the profile leaves the workload identity stable and forces every optimization to be disclosed as configuration.

The separation also maps onto how the suite is actually used. `min` supports installation, pre-lab checks, and continuous integration. `max` is the classroom reference contract whose assets, evaluator, and gate must not change, which is what makes two students' submissions comparable. `pro` exposes repetitions, backends, compilers, and ablations for projects or architecture studies, where the configuration is the experiment and comparability is scoped to whatever the record discloses.

7 Experimental Results

The evaluation tests three claims implied by the design. First, an unchanged upstream task can meet its quality gate through the laptop implementation. Second, the score-bearing suite fits a practical local execution budget. Third, the framework distinguishes an executable result from one with enough repeated evidence to support a timing claim.

The evidence campaign evaluates 12 canonical cases on an Apple M5 Max host from source commit `163d42ee3d`. CPU and Apple MPS are used according to the declared case contract. Every retained run passed execution, grading, provenance, source, and semantic checks. Where a case was measured more than once, its timing spread is reported alongside the median; a single measurement is reported as one measurement and carries no repeatability claim.

A single accepted run decides a quality verdict, while any timing claim rests on repeated measurement. That asymmetry only holds if the quality metric is genuinely deterministic. We measured whether it is. Each of 6 inference workloads was run under 5 seeds on the default MPS backend and then repeated once on CPU, giving 36 executions in total. The design is deliberately not a full crossing of seeds and backends; it tests seed sensitivity at depth on one backend and then checks whether the second backend agrees at all. The quality metric did not move under either test. The largest spread across seeds was 0.0e+00 and the largest difference between backends was 0.0e+00. Every value was bit-identical.

Training does not behave this way. Sweeping 3 seeds across the 2 training workloads cheap enough to repeat, the seed spread exceeded the margin to the effective threshold in both, and in both the verdict changed. Graph node classification is recorded as a pass at 0.7210 and falls to 0.7115 under another seed, below its tolerance floor. Time-series forecasting is recorded as a miss at 0.2924 and reaches 0.2889 under a third seed, inside its target. The verdict moves in both directions, so a single accepted run does not settle a training comparison.

We report this rather than averaging it away, and the suite is arranged so a student meets it. A production submission absorbs this variance into a many-run protocol, which is correct for publishing a number and unhelpful for learning why the protocol exists. Here the gap between a workload's seed spread and its distance to the gate is visible in the artifact, and reproducing it costs one command with a changed seed. A student who watches a passing workload miss, and a missing workload pass, has learned something about benchmark claims that no amount of prose about statistical rigor conveys. That is the intended use, and Section 8 builds an exercise on it.

The consequence for reading this paper is unchanged: a training row is one draw from a distribution about as wide as its distance to the gate, and should be read that way. Raising the accepted-run count for training

contracts would narrow the reported interval, but it would also remove the most direct demonstration the suite offers of why one run is not evidence.

This matters most for the cases sitting closest to their contract line. Text classification exceeds its target by roughly two parts in a hundred million and information retrieval matches its target to eight decimal places, margins thin enough to look like luck. They are not. Those workloads evaluate a pinned checkpoint over a fixed set and consume no randomness, so they reproduce the published value rather than approach it. The single-run acceptance rule is therefore sound for inference, and the result also shows that quality, unlike timing, does not depend on the execution backend.

The campaign uses a create-once policy. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, Low Power Mode, and sleep or wake changes are checked before and after each promotion execution. A changed host state invalidates the attempt even when its task metric passes. This separation is important because accuracy does not excuse unstable timing or uncontrolled measurement conditions.

7.1 Committed Reference Evidence

Figure 2 plots every one of the 14 contracts the suite executed against its own published target. Of those, 8 reproduce it and 6 fall short. Five of the six shortfalls are narrow, the widest under six percent relative, which is the expected spread when a contract is inherited rather than retuned.

Reinforcement learning is the exception and is a different kind of miss. Its adapter runs the pinned upstream reference, but the laptop configuration generates 16 self-play games per generation for four generations against a reference budget of 2,000 games per generation and up to 10,000 generations. The resulting value is far below the gate because the run is a bounded probe, not because the implementation diverges from the contract. It is plotted rather than hidden, since excluding an inconvenient result is precisely the discretion this framework is built to remove, but it should be read as a statement about budget and not about reproduction fidelity.

Of the executed contracts, 9 are admitted to score-bearing review, where 8 meet their inherited gates in every retained execution. The generator grades each retained result against the live registry contract, so a record graded under a superseded gate cannot publish its own looser bound; time-series forecasting is reported as a miss for exactly this reason. The score-bearing reference medians total 62.0 minutes for one score-bearing suite pass. The individual cases span 0.280 seconds to

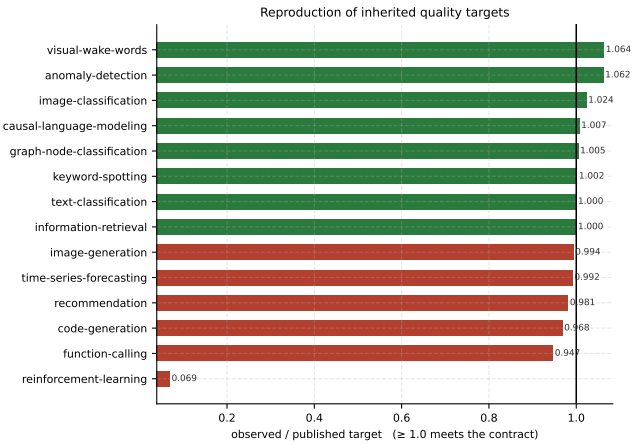

Figure 2. Reproduction of inherited quality targets. Each workload's observed value is divided by its own published target, so accuracy, ROC AUC, nDCG@10, hit rate, and MSE share one axis and a ratio of 1.0 is the contract line in every case. Direction is handled per metric, so a lower-is-better target such as test MSE is inverted before plotting. Most workloads sit within a few percent of the value published upstream, which is the expected result when a contract is inherited rather than retuned, and the bars below the line are recorded misses rather than relaxed targets. Reinforcement learning is the one wide bar and is not comparable to the others: it is a deliberately bounded self-play probe run at a small fraction of the reference game budget, so its distance from the line measures compute spent rather than fidelity lost.

35.1 minutes (Figure 3), which permits short classroom exercises

without pretending that every training workload is instantaneous. Image classification, keyword spotting, machine-sound anomaly detection, visual wake words, text classification, and information retrieval were each measured over repeated fresh processes, with timing coefficients of variation from 0.31% to 3.08%. Time-series forecasting reproduces the published PatchTST recipe to 0.2924 test MSE against an unchanged 0.2900 reproduction point, so it is retained as a miss; no tolerance was introduced after the fact to close a 0.0024 gap. The two retained causal-training measurements have a 5.19% timing coefficient of variation, narrowly above the unchanged 5% promotion limit.

Neither training shortfall is a budget artifact, which matters because the obvious objection to a laptop-scale suite is that its misses are really just truncated runs. The per-epoch curves in Figure 4 rule that reading out. Both workloads reach their best value and then move away from it, so the shortfall survives a longer run rather than closing with one. The recommendation contract makes the point concrete: an earlier configuration read a still-rising curve as an epoch limit, and extending it to twenty

Table 5. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions, so a value of one marks a case that carries no repeatability claim. None is an MLCommons-verified result.

Case	Role	Evidence (n)	Semantic Gate	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC $\geq$ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss $\leq$ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq$ 71.74%	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq$ 85.00%	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq$ 60.72%	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq$ 90.00%	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq$ 91.06%	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE $\leq$ 0.2900	0.2924 **(miss)**	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq$ 80.00%	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

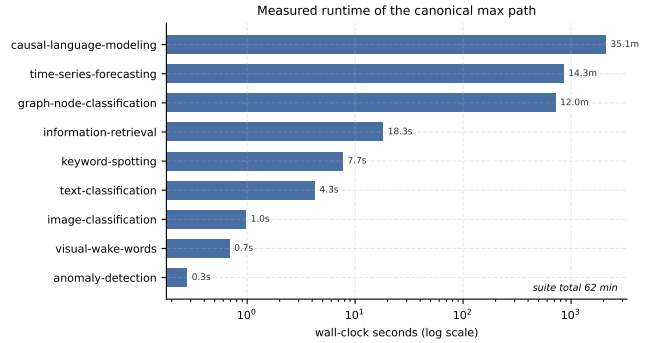

Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference laptop, on a log axis because the suite spans four orders of magnitude. The fixed-checkpoint inference workloads complete in seconds, while the workloads that train from scratch take minutes. The spread is what makes the suite usable in a class period: an instructor can assign the short end for a single session and reserve the training workloads for coursework that runs unattended.

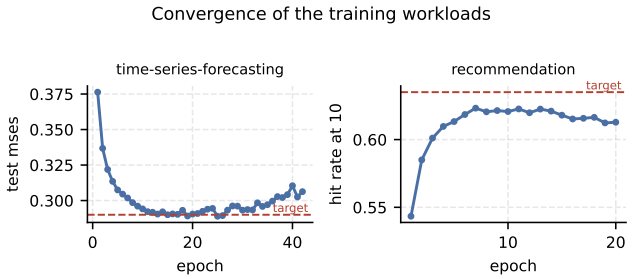

Figure 4. Convergence of the training workloads. Per-epoch task quality against the inherited target. Both panels show the same shape and it is not the one a longer run would fix. Time-series forecasting reaches the target line near epoch fifteen and then drifts back above it, and recommendation peaks at 0.6232 on epoch seven and declines thereafter. An earlier seven-epoch recommendation result read its still-rising curve as an epoch limit; extending the run to twenty epochs cost a further sixty-three minutes and returned a worse score, which is why the contract now caps the budget at the peak. Neither shortfall is a budget limit and neither was closed by relaxing a target.

epochs cost a further sixty-three minutes and returned a worse score.

Execution integrity is not adapter equivalence. The keyword-spotting record is valid measured evidence, but its committed three-resolver audit fails exact prediction parity. The quality-preserving but nonidentical adapter remains educationally useful while blocking promotion. The 90% quality gate is not a disagreement tolerance.

These results support the three evaluation claims stated at the start of this section, on a single machine. The inherited tasks remain reachable, the score-bearing reference path fits roughly one hour, and the Admission Rule keeps every case that missed its inherited gate out of

score-bearing review rather than promoting it alongside the cases that passed. They do not establish performance portability. Task metrics are intended to transfer within declared numerical tolerances; runtime is not. The table therefore identifies each device, while retained reports record the complete hardware and software fingerprint. A second machine should be expected to reproduce the gate and protocol, not the Apple M5 Max throughput.

7.2 Inference Latency

Inference evidence is eligible only after correctness and model identity are established. Causal full, prefill, and decode inference load one retained quality-passing training checkpoint and record its SHA-256 digest. The evidence importer verifies the checkpoint package, source report, source manifest, and package digest before accepting any phase. This closes a common loophole in which inference silently uses random, stale, or separately trained weights.

Full inference measures complete autoregressive requests. Prefill measures fresh KV-cache construction for a fixed 256-token prompt. Decode measures cached-token generation after a fixed prompt. Reports retain median, p90, and p99 latency together with phase-appropriate token throughput and sample counts. The three retained CPU measurements report 695.5 output tokens/s for full inference, 24.6 thousand prefill tokens/s, and 767.3 output tokens/s for decode.

Each phase passes its functional and lineage gate, but each currently has one retained execution. The rates are therefore provisional system observations, not repeatability evidence or portable performance targets. Their value lies in their checkpoint, protocol, sample, and hardware context.

8 Classroom Use

The benchmark contract is itself the teaching object. The intended progression has four stages. Students **reproduce** a fixed profile and verify its report and provenance manifest, **diagnose** where time is spent across data loading, model execution, and request scheduling, **optimize** one systems variable while preserving the declared quality gate, and then **defend** the resulting comparison against the evidence supporting it. The profiles map onto that progression directly. An instructor can use `min` as a pre-lab environment check, `max` as the fixed reference for a bounded assignment, and `pro` as the disclosure envelope for an open-ended project, while the workload identity and semantic gate stay fixed throughout. Optimization therefore changes the system under test, never the definition of success.

The **defend** stage has a concrete instrument, and it comes from the suite's own measured behavior. Two training workloads have a seed spread wider than their distance to the gate, so a student who reruns graph node classification under a different seed watches a passing result miss, and a student who reruns time-series forecasting watches a recorded miss pass. Both take one command and a changed seed. The exercise asks a question the reader of any benchmark table should ask: given this spread, which comparisons does this evidence actually support? A student who has seen a verdict move learns why a single number is not a claim, which is difficult to teach from a suite where every result is stable.

Three standalone labs make that progression concrete. One profiles a deliberately inefficient training loop and requires the submission to carry both configurations, task-quality outcomes, timing distributions, and an explicit statement of comparability. One implements a small inference system-under-test interface and compares naive autoregressive decoding against KV-cache decoding, passing only when both paths emit identical tokens for every measured query. One contrasts a dense model with a sparse mixture-of-experts model at similar headline parameter counts, reporting active parameter fraction alongside throughput to show why parameter count alone is not a systems cost model. Each lab has a deterministic offline smoke mode that exercises real forward, backward, inference, or parity paths. Those smoke paths establish software regression coverage rather than classroom efficacy. This draft claims no measured learning outcomes; course pilots, accessibility review, and instructor studies remain future work.

9 Limitations and Future Work

The framework admits only contracts that fit local hardware without changing their defining task. That choice preserves authority and access, but it does not capture every production bottleneck. Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid compar-

isons, preserving semantic gates, and defending systems claims.

Timing is reported from the measurements each case actually has, and a case measured once is reported as measured once. Promotion to a published baseline would still require repeated timing under stable host conditions, asset policy, and portable evidence, which v0.1 does not claim.

The current references come from one Apple M5 Max MacBook Pro with 64 GB of memory and 18 physical cores. Runtime is platform-specific, and every timing and repeatability number here characterizes that machine rather than the suite. Quality is better established: the determinism study shows the metric unchanged across seeds and across the MPS and CPU backends, so numerical drift between backends is measured rather than assumed for the inference cases. That leaves timing, training-workload seed sensitivity, and any drift specific to a different vendor's arithmetic as open questions. Cross-platform work should include Linux CPU and CUDA systems, clean-wheel installation, and repeated timing on each.

The paper argues that production suites exceed a course budget without measuring the alternative. A direct comparison, attempting an established production benchmark at reduced scale on the same laptop and reporting where it succeeds or fails, would test that premise rather than assert it. This is out of scope for v0.1 because the comparison is only meaningful against a submission-grade configuration of the other suite, which carries its own review and disclosure obligations. Until it is done, the accessibility argument rests on the operational requirements those suites document rather than on a measurement made here.

Several datasets remain fetch-only or require a release-policy decision. The project also needs an authoritative license for the complete published artifact, public hosting for larger evidence packages, and external decisions about the name and any relationship to MLCommons. Technical completion does not settle those governance questions.

Power-state guards improve the internal validity of timing campaigns but do not provide calibrated energy. Defensible joule claims require supported platform counters or an external meter. The five functional-stage additions now exercise recommendation, reinforcement learning, code generation, image generation, and function calling end to end, but they provide no authoritative quality or timing claim. Each must clear its unchanged quality contract before entering the promotion and repeatability protocol described in Section 7.

9.1 Threats to Validity

Internal validity. Where a case was measured over repeated fresh processes, that repetition reduces sensitivity to a single timing sample; where only one measurement exists, no repeatability claim is made and the report says so. Shared implementation error can remain across every case regardless of how many times it was run. Provenance digests detect byte changes and path drift; they do not prove that the computation occurred.

External validity. One laptop establishes local feasibility, not performance portability. The evidence records hardware and software fingerprints so later systems can test both quality transfer and timing variation.

Construct validity. The framework assumes that practicing controlled comparisons on reduced workloads improves production benchmarking judgment. That claim is plausible but unmeasured. The selection process may also favor tasks with compact pretrained checkpoints or established small evaluation sets. Formal course studies, independent workload review, and external artifact evaluation are needed to assess both effects.

10 Conclusion

MLPerf EDU brings professional benchmark discipline to a scale that students and individual researchers can inspect. Its central contribution is not a list of small models. It is an executable contract that binds model, data, hardware, scenario, quality, protocol, and evidence before a performance claim is accepted, and that takes every quality target from an upstream authority so the project retains no discretion to move a threshold it fails to reach.

The present implementation contains 14 workloads across 7 suites, of which 9 promotion-scope workloads carry 12 canonical evidence cases, while 5 additions have functional evidence only. In all, 8 of 9 score-bearing cases pass their quality gates, 1 is retained as a disclosed miss, and all performance-bearing phases pass their functional and lineage gates. Repeated timing, where collected, varies between 0.31% and 3.08%. Measuring the suite also produced a result about benchmark protocol rather than about any workload in it. Inference quality is bit-identical across seeds and across two execution backends, so a single run settles it, while both training workloads swept have a seed spread wider than their margin to the gate and flip verdict under a different seed. A protocol that accepts one run is therefore right for part of this suite and wrong for the rest, which is a distinction worth carrying into any small-scale benchmark. Those results make the benchmark ready for external technical review, not for a claim of official status. Public evidence

hosting, cross-platform reproduction, dataset policy, licensing, and MLCommons feedback remain visible next steps.

The intended educational value lies in the workflow students practice. They reproduce a baseline, diagnose a bottleneck, change one variable, preserve quality, and defend comparability with artifacts. That workflow can outlast any particular model or hardware generation. Demonstrating its learning effect is the next empirical question.

Reproducibility

The repository contains the executable registry, reference runners, evidence summaries, website, labs, tutorial, and paper generator. The public entry point is `mlperf`. Runs produce JSON, HTML, CSV, and `.provd.json` artifacts. The paper imports generated registry counts and reference values so its quantitative claims remain synchronized with the committed class-labeled evidence. The larger raw packages are retained for local reviewer handoff and do not yet have public URLs.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.

Berkeley Function-Calling Leaderboard. Berkeley Function-Calling Leaderboard V4, 2026. URL https://gorilla.cs.berkeley.edu/leaderboard. Accessed July 15, 2026.

Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL https://doi.org/10.1145/3352020.3352024.

F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015. doi: 10.1145/2827872. URL https://doi.org/10.1145/2827872.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 173–182, 2017. doi: 10.1145/3038912.3052569. URL https://doi.org/10.1145/3038912.3052569.

John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL https://doi.org/10.1145/1186736.1186737.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, et al. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*, 2024. URL https://arxiv.org/abs/2409.12186.

Andrej Karpathy. nanoGPT, 2022. URL https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291. Pinned upstream repository revision.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2206.00364.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.

Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, et al. MLPerf Training Benchmark. *arXiv preprint arXiv:1910.01500*, 2019. URL https://arxiv.org/abs/1910.01500.

Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL https://doi.org/10.1109/mm.2020.2974843.

MLCommons. MLPerf Inference Rules, 2026. URL https://github.com/mlcommons/inference_policies/blob/c547732b539cb3a14cc5680597714c8c1df4cad0/inference_rules.adoc. Accessed July 13, 2026.

Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091*, 2019. URL https://arxiv.org/abs/1906.00091.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=Jbdc0vTOcol.

Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL https://doi.org/10.1145/369275.369291.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. URL https://arxiv.org/abs/1910.01108.

Pete Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *arXiv preprint arXiv:1804.03209*, 2018.

Fanjia Yan. A function calling perspective on scalable large language model agent evaluation. Master's thesis, EECS Department, University of California, Berkeley, 2025. URL https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/EECS-2025-184.html.